

第 5 章：函数逼近、稳定性与 Deadly Triad

5.1 本章符号说明

符号/缩写	English	中文	含义
$V(s)$	State-value Function	状态价值函数	状态 s 的长期价值。
$Q(s,a)$	Action-value Function	动作价值函数	状态-动作对 (s,a) 的长期价值。
$\pi(a \mid s)$	Policy	策略	状态 s 下选择动作 a 的概率。
$\theta \mid w$	Parameters / Weights	参数 / 权重	策略网络或价值网络的可学习参数。
y	Target	训练目标值	Bellman target, 用来监督当前估计。
$L(w)$	Loss Function	损失函数	训练价值网络时最小化的目标。
behavior policy	Behavior Policy	行为策略	实际收集训练数据的策略。
target policy	Target Policy	目标策略	当前算法想学习或优化的策略。
on-policy	On-policy Learning	同策略学习	数据来自当前正在学习的策略。
off-policy	Off-policy Learning	异策略学习	用一个策略的数据学习另一个策略。
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似 $Q(s,a)$ 的 value-based 算法。
TD	Temporal Difference	时序差分	用一步 reward 和下一状态估计构造 target 的方法。
RL	Reinforcement Learning	强化学习	本章讨论 RL 从表格方法进入函数逼近后的稳定性问题。
LLM	Large Language Model	大语言模型	文中作为高维状态或复杂序列系统的例子。

5.2 本章目标

本章从表格型 RL 过渡到深度强化学习。你需要掌握：

- 为什么需要函数逼近。
- 线性函数逼近和神经网络逼近。
- value function approximation。

- bootstrapping 的影响。
- off-policy learning 的分布问题。
- Deadly Triad。

5.3 本章主线

前面几章默认可以为每个状态或状态-动作对存一个表格值。真实任务里状态空间通常巨大或连续，所以必须用参数化函数近似 $V(s)$ 、 $Q(s,a)$ 或 $\pi(a|s)$ 。本章先建立函数逼近的训练形式，再解释为什么 bootstrapping、off-policy learning 和 function approximation 同时出现时会带来稳定性问题。

5.4 本章新增概念说明

本章开始进入 deep RL，先把稳定性相关名词放在一起：

名词	中文	先记住什么
function approximation	函数逼近	用参数化模型近似 $V(s)$ 、 $Q(s,a)$ 或 $\pi(a s)$ ，不再为每个状态单独建表。
value function approximation	价值函数逼近	用模型预测价值函数，例如 $Q(s,a;w)$ 。
bootstrapping	自举	target 里用了模型当前自己的估计，例如 $r+\gamma V(s')$ 。
off-policy distribution shift	异策略分布偏移	数据来自一个策略，学习目标却是另一个策略，导致训练分布和目标分布不一致。
Deadly Triad	致命三角	function approximation、bootstrapping、off-policy 同时出现时，训练容易不稳定。
replay buffer	经验回放池	存历史 transition，训练时随机采样，打破连续样本相关性。
target network	目标网络	滞后更新的网络，用来让 Bellman target 更稳定。
overestimation bias	过估计偏差	max 操作容易选择被噪声高估的动作，导致 Q 值偏高。
reward clipping / gradient clipping	奖励截断 / 梯度截断	工程稳定化技巧，用来限制训练信号或梯度过大。

5.5 函数逼近为什么出现

5.5.1 为什么需要函数逼近

表格型方法为每个状态或状态动作对存一个值：

$$V(s)$$

$$Q(s, a)$$

但真实任务中状态空间可能极大，甚至连续：

- 图像输入。
- 机器人连续状态。
- 推荐系统用户特征。
- LLM 对话状态。

这时无法为每个状态单独建表，需要用参数化函数近似价值函数或策略。

例如：

$$V(s) \approx V(s; w)$$

$$Q(s, a) \approx Q(s, a; w)$$

$$\pi(a | s) \approx \pi(a | s; \theta)$$

5.5.2 Value Function Approximation

用神经网络近似 Q 函数：

$$Q(s, a; w)$$

训练目标通常来自 Bellman target。

例如 DQN 中：

$$y = r + \gamma \max_{a'} Q(s', a'; w^-)$$

损失：

$$L(w) = \mathbb{E}[(y - Q(s, a; w))^2]$$

其中 w^- 是 target network 参数。

5.5.3 函数逼近的好处

- 可以处理高维状态。
- 可以泛化到没见过的状态。
- 可以结合深度学习特征提取能力。
- 能用于图像、文本、连续控制等任务。

5.5.4 函数逼近的风险

表格型方法中，不同状态动作对的值相互独立。

函数逼近中，共享参数会导致一个样本更新影响许多状态动作对的估计。

这带来几个问题：

- 训练目标非平稳。
- 样本之间高度相关。
- 误差可能通过 bootstrap 扩散。
- 价值函数可能过估计或发散。

5.6 不稳定来源：Deadly Triad

5.6.1 Bootstrapping

Bootstrapping 指 target 中包含当前模型自己的估计。

例如 TD target：

$$r + \gamma V(s')$$

Q-learning target：

$$r + \gamma \max_{a'} Q(s', a')$$

它的优点：

- 可以在线更新。
- 不需要完整 episode。
- sample efficiency 较高。

它的风险：

- target 依赖当前估计。
- 估计错误会被继续传播。
- 函数逼近下更容易放大不稳定性。

5.6.2 Off-policy Distribution Shift

off-policy 学习中，数据来自 behavior policy，但学习目标是 target policy。

data distribution: $d_{\mu}(s, a)$

target policy: π

如果 behavior policy 产生的数据分布和 target policy 需要的数据分布差异很大，模型可能会在没数据覆盖的状态动作上产生错误估计。

这种问题在 offline RL 中尤其严重。

5.6.3 Deadly Triad

Deadly Triad 指三个因素同时出现时，RL 训练可能不稳定甚至发散：

1. Function approximation。
2. Bootstrapping。
3. Off-policy learning。

为什么危险？

- 函数逼近会让误差泛化到其他状态。
- bootstrapping 会用当前估计构造 target，错误会自我强化。
- off-policy 会带来数据分布和学习目标不一致。

DQN 同时包含这三个因素，因此需要额外稳定技巧：

- replay buffer 打破样本相关性。
- target network 稳定 bootstrap target。
- reward clipping。
- gradient clipping。
- Double DQN 降低 overestimation bias。

5.7 DQN 稳定化思想

5.7.1 从 Deadly Triad 到 DQN 稳定化

下面的 replay buffer 和 target network 不应该理解成孤立技巧，而是第 6 章 DQN 的铺垫。逻辑是：

```
Q-learning + neural network
  -> function approximation
  -> bootstrap target
  -> off-policy replay data
  -> Deadly Triad risk
  -> replay buffer / target network / Double DQN
```

所以第 5 章先解释“不稳定从哪里来”，第 6 章再系统讲“DQN 如何把这些稳定化设计组合成算法”。

5.7.2 Experience Replay 的稳定作用

Replay buffer 存储历史 transition：

```
(s, a, r, s', done)
```

训练时从 buffer 中随机采样 mini-batch。

好处：

- 打破连续样本相关性。
- 提高样本利用率。
- 让数据分布更平滑。

限制：

- 数据可能 stale。
- 对 on-policy 算法不天然适用。
- 分布偏移仍然存在。

5.7.3 Target Network 的稳定作用

如果 target 和 online prediction 使用同一个网络：

$$y = r + \gamma \max_{a'} Q(s', a'; w)$$

那么模型参数一更新，target 也立刻变化，训练目标非常不稳定。

Target network 使用滞后的参数 w^- ：

$$y = r + \gamma \max_{a'} Q(s', a'; w^-)$$

每隔一段时间同步：

$$w^- \leftarrow w$$

或者软更新：

$$w^- \leftarrow \tau w + (1 - \tau) w^-$$

5.8 本章例子：为什么表格法不够

5.8.1 例子 1：Atari 像素输入

Atari 游戏状态是一张图片。如果用表格法，每一种像素组合都对应一个状态，状态数量几乎无限。

函数逼近的做法是：

输入：游戏画面
神经网络：CNN / encoder
输出：每个动作的 Q 值

这样模型可以泛化。例如两个画面只差几个像素，但局面本质相似，网络可以给出相近价值。

5.8.2 例子 2：推荐系统高维状态

推荐系统状态可能包含：

- 用户画像。
- 最近点击序列。
- 商品特征。
- 时间、地点、设备。

这些状态无法枚举。函数逼近可以把高维特征映射到价值或策略：

$$Q(s, a; \theta)$$

5.8.3 例子 3：Deadly Triad 为什么危险

假设用 DQN 训练一个推荐策略：

- function approximation: 用神经网络估计 Q 。
- bootstrapping: target 里有 $\max_a Q(s', a)$ 。
- off-policy: 数据来自旧策略或 replay buffer。

如果新策略常推荐训练数据里很少出现的内容, Q 网络可能对这些动作过度乐观; bootstrap 又会把这种高估继续传播, 训练就可能发散。

面试表达:

Deadly Triad 的危险不是三个词各自危险, 而是它们叠加后, 错误估计会通过 bootstrap 在 off-policy 分布外被不断放大。

5.9 本章面试高频问题

5.9.1 为什么表格型 RL 不够用?

因为真实任务的状态空间通常很大或连续, 无法为每个状态动作对单独存一个值。函数逼近可以泛化到未见状态, 并处理高维输入。

5.9.2 什么是 Deadly Triad?

Function approximation、bootstrapping 和 off-policy learning 三者同时出现时, 强化学习训练容易不稳定甚至发散。

5.9.3 为什么神经网络加 Q-learning 会不稳定?

Q-learning 使用 bootstrap target, target 又依赖当前估计。神经网络共享参数, 一个样本更新会影响大量状态动作估计。再加上 off-policy 数据分布偏移, 误差可能被放大。

5.9.4 replay buffer 解决什么问题?

它通过随机采样历史 transition, 打破连续样本相关性, 提高样本利用率, 并让训练数据分布更平滑。

5.9.5 target network 解决什么问题?

它让 Bellman target 在一段时间内保持相对稳定, 避免 online network 更新时 target 也同步剧烈变化。

5.10 本章练习

1. 用自己的话解释 bootstrapping。
2. 画出 DQN 中 online network 和 target network 的关系。
3. 解释为什么 off-policy + function approximation 容易出问题。
4. 总结 DQN 针对 Deadly Triad 做了哪些稳定化设计。

[章节索引](#)

[← 上一章](#) [下一章 →](#)